



Enhanced intrusion detection framework for securing IoT network using principal component analysis and CNN

Abdul Mazid^a, Sheeraz Kirmani^b, and Manaulah Abid^a

^aDepartment of Electrical Engineering, Jamia Millia Islamia, India; ^bDepartment of Electrical Engineering, Aligarh Muslim University, Aligarh, India

ABSTRACT

The Internet of Things (IoT) has transformed our world by connecting smart devices and enabling seamless interactions. This reliance, however, has led to new security issues and types of attacks. It is of the utmost importance to safeguard the security of IoT networks, with network intrusion detection systems (NIDS) having a significant impact. This paper proposes a novel approach integrating Principal Component Analysis (PCA), Pearson Correlation Coefficient (PCC), and Convolutional Neural Network (CNN) to overcome these security issues. Our innovative method reduces data dimensionality and selects highly correlated features using PCC and PCA, addressing overfitting and improving model performance while maintaining high computational speed and low costs. Our approach uniquely distinguishes between benign and threat packets by employing 1D-CNN, 2D-CNN, and 3D-CNN algorithms trained on Edge-IIoTset and NSL-KDD benchmark datasets. The findings from our experiments indicate that the proposed framework significantly enhances accuracy, precision, recall, and F1-score compared to existing models for both binary and multiclass classifications. Our binary classification models achieved exceptional performance, with an average accuracy of 99.76%, 99.79% precision, 99.89% recall, and 99.85% F1-score on the Edge-IIoTset dataset. On the NSL-KDD dataset, the models attained 99.20% accuracy, 98.07% precision, 97.95% recall, and 97.71% F1-score. For multiclass classification, the proposed model demonstrated an average accuracy of 99.41%, precision of 98.61%, recall of 98.49%, and an F1-score of 98.56% on the Edge-IIoTset dataset. On the NSL-KDD dataset, the model achieved 92.43% accuracy, 93.21% precision, 93.60% recall, and a 93.7% F1-score. Our research introduces a significant advancement that substantially improves NIDS capabilities, making IoT networks safer and more connected.

KEYWORDS

Deep learning; internet of things; intrusion detection; PCA; Security

1. Introduction

The IoT is a rapidly expanding technology that connects smart things to the internet, resulting in a more interconnected and intelligent society (Sufyan & Banerjee, 2020). The proliferation of IoT technology has substantially grown in the past 10 years, owing to the integration of internet connectivity and computational proficiencies into a wide range of smart devices, sensors, and objects (Chettri & Bera, 2020). IoT has transformed many industries, including transportation, healthcare, agriculture, and smart cities (Kirmani et al., 2022), by connecting billions of smart devices to the internet (Abiodun et al., 2021). However, IoT integration also introduces security vulnerabilities as a bridge between the digital and physical worlds (Khraisat et al., 2019). Even while IoT systems have significantly enhanced people's lives, maintaining their security is still a big

concern. Protecting IoT devices from threats can be challenging due to the need for uniform protocols in their creation. Furthermore, using multiple communication protocols in IoT networks adds complexity, making it difficult to implement a consistent and effective security strategy (Meffert et al., 2017). Unfortunately, security flaws in IoT protocols might expose networks to man-in-the-middle (MITM) attacks such as data theft, disruption, and device interruption (Firdous et al., 2017).

An Intrusion Detection System, also known as an IDS, is a crucial component of network security that observes and analyses network traffic to detect malicious behavior or unauthorized access. IDS is intended to detect potential security breaches and warn security analysts or administrators, allowing them to make suitable steps to prevent the effects of security incidents. There exist two distinct

classifications of IDS: signature-based and anomaly-based IDS. Signature-based IDS utilizes pre-established patterns or rules associated with known assaults to detect unauthorized intrusions. In contrast, anomaly-based IDS uses statistical models or Machine learning (ML) algorithms to identify variances from typical network performance, which may suggest the presence of probable anomalies. Including IDS as part of a complete security plan to protect against cyber threats is critical. IDS are specifically engineered to identify and flag instances of unauthorized access that have successfully bypassed the preexisting security mechanisms. Consequently, the implementation of IDS improves the overall security of IoT networks. Anomaly-based IDS establishes a usual response pattern and categorizes any behaviors that vary from this pattern as suspicious activities, thus enhancing the general safety of the system (Han et al., 2023; Pawar & Sachdeva, 2022; Wang et al., 2023). An IDS identifies network intrusions by monitoring all packets and determining whether any incoming or outgoing packets are compromised. Traditional intrusion detection models relied on statistical and knowledge-based methods, which often struggled with detecting unknown attacks and managing large volumes of network traffic. ML presents a powerful solution, offering the capability to develop various strategies that strengthen the security of systems like cloud computing, IoT, and other network-based systems. One significant application of machine learning is in the development of an IDS, where the system analyzes network traffic, differentiates between normal and abnormal behavior, and categorizes them accordingly (Srilatha & Shyam, 2021; Xu et al., 2018).

Numerous conventional and ML based approaches have been used for NIDS. However, it has been demonstrated that these methods could be more suitable for industrial use and efficient at quickly identifying attacks (Li et al., 2022). Conventional ML models rely on pre-defined characteristics and techniques, whereas Deep Learning (DL) can comprehend complex patterns and adjust to new attack patterns. This adaptability makes DL more suitable for processing IoT networks' massive amounts of data. DL is a more effective technique for overcoming the limitations of ML algorithms (Hussain et al., 2020; Jayalaxmi et al., 2022). In

contrast to traditional ML, DL allows for a more thorough representation of raw data and enhanced classification or prediction accuracy by employing numerous layers in a computational model. DL is an effective method for detecting network intrusion, with multiple studies reporting improved detection accuracy and fewer false positives than traditional methods (Imran et al., 2022). Several DL approaches also have been employed for network intrusion detection purposes. However, the utilization of DL in NIDS poses certain difficulties, such as the necessity of significant volumes of labeled data for model training and the potential vulnerability to adversarial attacks, where attackers can change the data to evade detection (Al-Garadi et al., 2020). Given the substantial computing power and vast amounts of data collected from interconnected devices, DL models are well-suited to enhancing IoT security, particularly in intrusion detection, analyzing user behaviors, addressing vulnerabilities, and preserving privacy. DL techniques, especially CNNs, excel at learning, extracting, and identifying complex features and patterns directly from raw IoT data, thereby enhancing the device's ability to effectively detect potential threats and attacks within the IoT environment. Additionally, DL models outperform traditional ML methods in automatic feature extraction, eliminating the need for manually crafted statistical features (Wei et al., 2020).

CNNs are successful in network intrusion detection as they can recognize patterns in data using convolutional layers. Their ability to identify spatial patterns in network traffic data makes them appropriate for identifying intrusions. Although other neural network architectures like RNNs and DBNs may be better suited for different tasks, CNNs have proven successful in network intrusion detection tasks requiring pattern recognition. PCA is frequently utilized to lower the dimensionality of input data for CNNs, enabling faster training by decreasing computational complexity. It achieves this by mapping the input data to a space of lower dimensions, where the new principal components selected to explain the maximum variance in the data can help CNNs avoid overfitting and increase accuracy by removing noise and unnecessary features. PCA can transform data into uncorrelated components,

making it easier to learn non-linear relationships. Using PCC along with PCA can identify highly correlated features, which can be removed to reduce overfitting and improve model performance on non-linear data. Consequently, integrating PCA can enhance CNNs' performance in network intrusion detection tasks (Guo et al., 2021; Liu & Durlinsky, 2021; Mazid et al., 2023).

This study aims to improve the performance and minimize the model complexity of existing ML-based IDS methods. The proposed network utilizes an efficient feature selection technique to recognize significant features and lower the feature dimensions. These strategies greatly improve network accuracy and reduce model complexity. Our paper presents a novel intrusion detection method that utilizes a DL approach, combining CNN and PCA. The PCA technique is employed along with PCC to select important features, reduce dimensionality, and provide the CNN models with a new encoded dataset. The integration of PCA and PCC within our framework facilitates improved feature representation, enhanced generalization, and efficient model training, thereby contributing significantly to the total performance of our intrusion detection system for securing IoT networks. Three different CNN models (CNN1D, CNN2D, and CNN3D) are trained using the encoded data in our experiment. We employ binary and multiclass classification for model training. The primary aim of this study is to present a model with a higher detection rate and accuracy. Through our experimentation, our proposed approach can address the challenges of another state-of-the-art method and improve intrusion detection performance.

1.1. Contributions

The key contributions of this study can be outlined as follows:

- The paper proposes a PCA-CNN-based framework to detect intrusions in IoT networks.
- The proposed framework uses PCC techniques to identify the highly correlated features in the given dataset and then PCA to reduce the dimensionality. Hence, we can solve the overfitting problem, interoperability, which

improves the model performance while keeping computation high speed and costs low.

- The encoded data is then reshaped into dimensions (1D, 2D, and 3D), which can be fed to the CNN models for training and validation.
- The proposed framework is evaluated on Edge-IIoTset and NSL-KDD datasets and compared with previous studies. The findings from our experiments indicate that the proposed framework outperforms previous approaches regarding accuracy, precision, recall, and F1-measure.
- The proposed framework also requires less time for testing. It achieves a higher detection rate and accuracy than previous works, demonstrating that it is better suited for IoT use cases where latency is critical.

1.2. Organization of the paper

In this paper, the following structure is followed: [Section 2](#) gives an outline of the existing work on intrusion detection in IoT networks. [Section 3](#) explains our proposed approach and its components. [Section 4](#) outlines the experimental setup, the results of our evaluation, and the ensuing discussion. Finally, the paper concludes in [Section 6](#) by recapping the results and identifying potential paths for future research.

2. Literature review

Researchers in the field of IDS have explored various DL and ML-based approaches. Li et al. (2020) presented a fusion approach for intrusion detection in IoT environments using a multi-CNN architecture, attaining high precision and low complexity. Javeed et al. (2023) suggested a hybrid model combining bidirectional GRU, LSTM, and a softmax classifier to detect network edge attacks. This IDS utilizes Truncated Backpropagation Through Time (TBPTT) for faster learning with long sequences of network data and suggests a framework suitable for challenging Situational Awareness (SA) environments. Additionally, Popoola et al. (2021) utilized LSTM autoencoder (LAE) and deep bidirectional LSTM (BiLSTM) algorithms to

detect botnet attack traffic effectively. Their method reduced the dimensionality of traffic data using LAE and distinguished botnet attack traffic with deep BiLSTM. Advanced methodologies have been introduced to enhance network traffic classification through DL. A meta-learning-based method for network traffic classification was introduced by Xu et al. (2020). They utilized an FC-Net to detect intrusions in networks with a few samples, combining feature extraction and a similarity network. Ning et al. (2022) suggested a few-shot MTC technique founded on semi-supervised learning, transfer learning, and domain adaptation to achieve high-performance classification of unlabeled traffic data. Zhao et al. (2022) presented a lightweight IDS based on a novel lightweight deep neural network (DNN) architecture to implement DL models on resource-constrained IoT devices. Their experimental results demonstrated excellent performance in intrusion detection while maintaining a small size and low complexity. Senthil and Muthukumaran (2023) introduced a GAN model incorporating multi-kernel deconvolution and three levels of skip connections to address data class imbalance. The model also includes a Traffic Encoder loss and a specialized generator for network traffic. JBiRSA with GAN has distinguished between normal and threat IoT attack traffic data. Ding et al. (2023) presented DeepAK-IoT, a powerful DL model for detecting cyberattacks on IoT devices. This approach achieved an impressive 94.96% accuracy on the Edge-IIoTset dataset, making it an effective solution for managing cyber threats in IoT networks. A malware detection approach based on CNN and LSTM was proposed by Yan et al. (2018), using grayscale images of a malware file's opcode sequence. This method outperformed alternative ML methods such as SVM, RF, and linear k-nearest neighbor. Gómez and Muñoz (2023) developed a classification model that employs DL techniques to identify malware in Android APKs. Their research shows encouraging outcomes in detecting Android malware. Djaidja et al. (2024) proposes an early flow classification in NIDS using a DL model based on RNNs, LSTM and GRU, with attention mechanisms. This approach effectively exploits the sequential

nature of network packets, improving the efficiency of early flow classification.

Fatani, Abd-Elaziz, et al. (2021) proposed a feature extraction approach using CNN along with a new feature selection method based on the Transient Search Optimization Algorithm (TSO), improved with operators from the Differential Evolution (DE) algorithm to improve the balance between exploitation and exploration. In another study, Fatani, Abd-Elaziz, et al. (2021) developed innovative feature extraction and selection methods for an IDS that utilizes Swarm Intelligence (SI) algorithms and CNNs. They introduced a new feature selection technique using the Aquila Optimizer (AQU) SI algorithm. Alqahani and Khan (2023) presented an optimal hybrid TS-Cascade R-CNN model for cyberattack detection, capable of recognizing and classifying intrusions, including sophisticated attacks. Their experimental analysis demonstrated a high accuracy rate of 99.2% across various metrics. Bhandari et al. (2023) presented an AI-based approach to identify malware attacks in diverse scenarios. Their performance testing of a deployed DNN model on IoT gateways showed promising results, with minimal memory usage (0.42 GB for NVIDIA Jetson and 0.2 GB for Raspberry Pi) and a modest increase in power consumption (average 13.5% per device).

Muñoz et al. (2022) explored the impact of attacks on communication protocols such as WiFi, Bluetooth-LE, GPS, 433 MHz, and NFC in local settings. They identified vulnerabilities and suggested countermeasures to improve security. Gómez and Muñoz (2023) applied a supervised learning approach for Android malware detection using a dataset of over 18,000 samples categorized into Adware, Banking, SMS, Riskware, and Normal. They analyzed static malware features and developed an Android app to evaluate their algorithm, confirming its effectiveness in real-world scenarios. Rani and Gagandeep (2022) proposed a DNN that addresses class imbalance through a classifier-level solution. Data is converted and normalized before being fed into the network, where a weighted cross-entropy function manages the imbalance during training. Imrana et al. (2021) presented a BiDLSTM model based on LSTM and RNN for identifying network intrusions. However, the BiDLSTM technique exhibited

limitations, such as high complexity and longer training time than the typical LSTM model. Almotairi et al. (2024) introduced a heterogeneous ML-based stack classifier model for IoT intrusion detection, employing feature selection and ensemble modeling to improve classification metrics for detecting intrusions in IoT data. Dahou et al. (2022) proposed a feature extraction and selection model, integrating DL with Metaheuristic (MH) optimization algorithms. They used the Reptile Search Algorithm (RSA) to choose the most favorable feature subset, reducing data dimensionality and improving classification performance. Ling and Hao (2022a) introduced an intrusion detection model, NMAIFS MOP-AQAI, which reduces high-dimensional features for quick detection using advanced techniques. Kothai and Poovammal (2024) proposes an IDS using hybrid CNN-GRU mechanisms to detect anomalies in vehicular networks, addressing overfitting and low-velocity issues. Their model enhances attack detection and protection, outperforming traditional IDS by 10.79% in performance.

Similarly, their NMIFS MOP-AQGA algorithm enhances intrusion detection speed, adaptability, and accuracy by selecting optimal features and training a classifier. Both models significantly improve detection accuracy and performance. Zhang et al. (2021) presented an IDS named the pretraining Wasserstein generative adversarial network intrusion detection system (PWGIDS) to solve the data imbalance issue. Mishra et al. (2011) discuss the DDoS problem, attack tools, principles, key features of an effective defense system, and the classification of existing defense mechanisms.

The above literature review highlights recent works in securing IoT networks and intrusion detection. However, these works face challenges such as limited accuracy and scalability. Our proposed approach combines PCA for dimensionality reduction, PCC for feature selection, and three CNN models (1D, 2D, and 3D) for training and validation to address these challenges. This integrated framework aims to improve intrusion detection performance and overcome the drawbacks of previous studies. Table 1 presents recent IDS research, including methods, advantages, limitations, and a comparison with our proposed approach.

3. Proposed methodology

The proposed approach aims to identify multiple types of intrusions to protect the security of IoT networks. The prime objective is to address network security issues with the IoT devices used in smart home environments. To achieve this, we employed a DL-based methodology to detect network intrusions. The proposed method thoroughly evaluates all data engineering parameters before validation with DL models.

As shown in Figure 1, a workflow of the proposed method for intrusion detection is presented. The various phases of our approach are discussed in the following subsections.

3.1. Data Preprocessing

Data preprocessing is essential to improve the data quality before feeding it into the proposed model. It includes data cleaning, dealing with missing values, removing irrelevant features, scaling features, etc. Many ML-based techniques are also involved in the preprocessing phase to deal precisely with relevant instances and their attributes (Jamalipour & Murali, 2022). In the data preprocessing, we identified and removed irrelevant columns such as IDs and timestamps that do not contribute to the model's predictive power. We addressed missing values by first locating them using *pandas.isnull()* and then imputing them to ensure dataset completeness and mitigate bias. For numerical features, missing values were replaced with the median of the respective columns using *pandas.DataFrame.fillna()*, while for categorical features, they were replaced with the most frequent value (mode) using the same method. Categorical variables were handled via label encoding for those with a small number of unique values using *sklearn.preprocessing.LabelEncoder*, and one-hot encoding was applied to those with a larger number of unique values using *pandas.get_dummies()* to optimize them for ML algorithms. We normalized or standardized numerical features to guarantee they are on the same scale using *sklearn.preprocessing.StandardScaler*. While we did not use data augmentation due to the adequacy of our dataset, we recognize its value for enhancing model generalization, especially in smaller datasets. The preprocessing pipeline ensured data integrity and

Table 1. Comparison of the proposed approach with recent studies.

Reference	Dataset	Strategy	Classifier	Metrics	Advantages	Limitations
Javed et al. (2023)	Edge-IoTset ToN-IoT	This paper proposed a hybrid approach merging BiGRU, LSTM and softmax classifier to detect network attacks.	BiGRU LSTM RNN	Accuracy F1-score, precision, Recall, ROC TPR	Efficient learning technique, better accuracy, Data privacy	Generalizability Resource constraints, Complexity
Bhandari et al. (2023)	Edge-IoTset	This paper presents a novel AI-enabled approach detects attacks and malware on IoT devices by live tracking streamed network traffic.	Decision Tree, Random Forest, SVM, ANN, DNN	Accuracy, F1-score, precision, Recall	Relevance to Modern Threats, Efficiency, High accuracy,	Lack of Comparative Analysis, scalability, Security
Rani et al. (2022)	NSL-KDD	This paper proposes a DNN that addresses class imbalance using data normalization and a weighted cross-entropy function during training.	DNN CS-NN	Accuracy F1-score precision Recall AUC	Better performance, Improved preprocessing, Effective attack detection	Complex Learning Task, Computational Overhead
Barnard et al. (2022)	NSL-KDD	This paper introduced XGBoost to achieve supervised intrusion detection, and for training an auto-encoder used for classification.	XGBoost, PCA, Autoencoder	Accuracy Precision Recall	better efficiency and reliability	Less accuracy, limited attacks classification
Alqahani and Khan (2023)	Edge-IoTset	This paper proposes a hybrid TS-Cascade R-CNN for cyberattack detection, combining transit-search and cascade regional CNN approaches.	CNN, Hybrid transit search	Accuracy F1-score precision Recall	High Accuracy, Sophisticated Attack Recognition, Identification of Vulnerabilities	Scalability, Resource constraints, Real-world Implementation Challenges
Imrana et al. (2021)	NSL-KDD	This paper discussed bidirectional LSTM based DL approach for intrusion detection	LSTM, BiLSTM	Accuracy precision recall, F1-score, FAR	Reduced false alarm rate, higher detection accuracy	Long Training Times, Data Imbalance
Fatani, Dahou, et al. (2021)	CIC2017, NSL-KDD, Bot-IoT, KDD99	This paper proposed feature extraction and selection incorporating CNN and Aquila Optimizer for IDS	CNN, AQU, PSO, BAT	Accuracy Precision Recall, F1-Score	Better accuracy, Adaptability to Dynamic Environments	Imbalanced Data, Vulnerability to Adversarial Attacks
Gómez and Muñoz (2023)	CICMal-Droid2020, CICMal-Droid2017, AndMal2017	This study introduces a DL approach for detecting and classifying attacks on Android devices.	Decision Tree, Random Forest, SVM	Precision Recall, AUC Accuracy, F1-Score, ROC	Enhanced Performance, Versatility and Autonomy	Computational Complexity, Data Integrity, Scalability
Almotairi et al. (2024)	ToN-IoT	The paper introduces a ML model for IoT intrusion detection, using feature selection and ensemble methods to improve classification performance.	Random Forest, Naïve Bayes, SVM, KNN	Accuracy Precision Recall, F1-score	Computational complexity, Improved model performance, Robust Security	Dataset Dependency, Computational Overhead, Model Interpretability, Scalability
Dahou et al. (2022)	KDDCup-99	This study proposes an effective IDS model with advanced DL and MH algorithms.	CNN, RSA	Accuracy Precision Recall, F1-score	Enhanced Performance, Strong metrics, Efficient Feature Extraction	Convergence Speed, Complexity, Security
Fatani, Dahou, et al. (2021)	KDDCup-99, NSL-KDD, Bot-IoT, CICIDS-2017	This paper presented an IDS framework using CNN and improved TSO.	CNN, TSO, MH	Accuracy Precision Recall, F1-score	Improved classification accuracy, Data privacy and security	Complexity and Computational Resources, Interpretability
Proposed Methodology	Edge-IoTset NSL-KDD	This paper presented a CNN-PCA based IDS for enhancing network security.	CNN PCA	Accuracy Precision F1- score, Recall, AUC ROC	High detection accuracy, Real- Time Potential, Privacy and Security, Data Robustness	Interoperability Challenges, Real-world Implementation Challenges

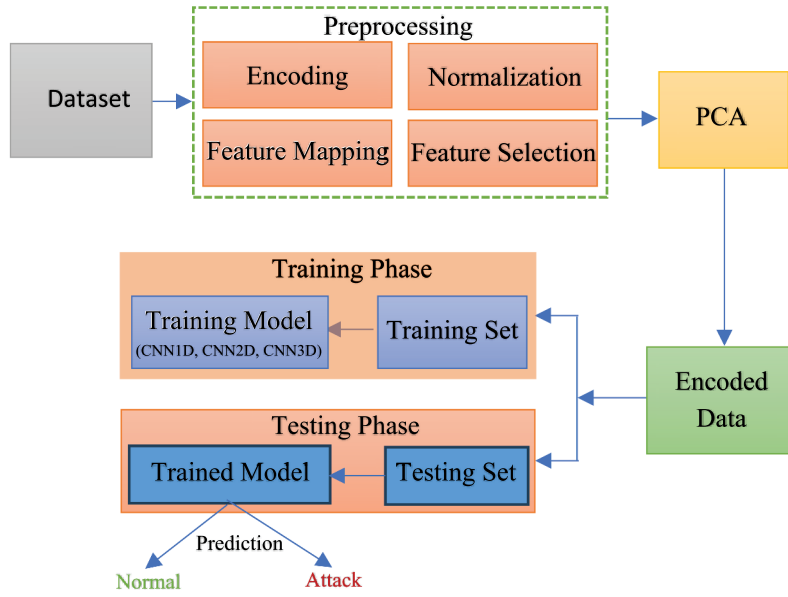


Figure 1. Workflow diagram of proposed IDS.

reliability, crucial for model performance. The study uses PCC statistical methods for feature selection. Feature Selection method chooses relevant features with the highest correlation level with the target parameter. This approach reduces computational complexity and improves execution speed. Algorithm-1 illustrates the process of data preprocessing and preparation, whereas Algorithm-2 demonstrates the data reshaping for CNN models. The flowchart of data preprocessing steps is shown in Figure 2.

In our approach, PCC is used with PCA to address the issue of non-linearity. By assessing the linear relationship between variables, the coefficient helps identify dependencies among principal components, providing insights into the data and potential non-linear relationships within the dataset. The transformation of the dataset into a new encoded dataset using PCA consists of three steps: feature mapping, feature selection, and then feature transformation. Feature mapping and feature selection are two important steps in data preprocessing. Feature mapping transforms categorical variables into numeric values better suited for machine learning techniques. Feature selection chooses the most appropriate features from a dataset, enhancing the model's performance. Our proposed framework uses a PCC for feature selection. The PCC measures the statistical

relationship among two variables. The framework chooses the N features with the highest PCC values (Venkatesh & Anuradha, 2019). The PCC is calculated for two random features, f_1 and f_2 , using Equation (1).

$$PCC(f_1, f_2) = \frac{\sum_{i=1}^n (x_i - \bar{f}_1)(y_i - \bar{f}_2)}{\sqrt{\sum_{i=1}^n (x_i - \bar{f}_1)^2} \sqrt{\sum_{i=1}^n (y_i - \bar{f}_2)^2}} \quad (1)$$

Equation (1) calculates the PCC between two features, f_1 and f_2 , using data points x_i and y_i . The absolute mean $\bar{f}_1$ and $\bar{f}_2$ is calculated as $|\frac{1}{n} \sum_{i=1}^n x_i|$ and $|\frac{1}{n} \sum_{i=1}^n y_i|$. The PCC value ranges from -1 to 1 , indicating the degree of linear dependence between the features. This PCC-based feature selection method aims to identify the level of reliance between the features. The resulting PCC value is then used as input to PCA for encoding. Third, the PCA approach transforms the collected optimized characteristic into a new dimension while maintaining the data's privacy. PCA is a statistical method that transforms the given data into a set of linearly uncorrelated features using an orthogonal transformation to minimize information loss. The output of this process is a newly encoded transformation referred to as the principal components. In our proposed framework, the obtained components are ordered based on their potential variance,

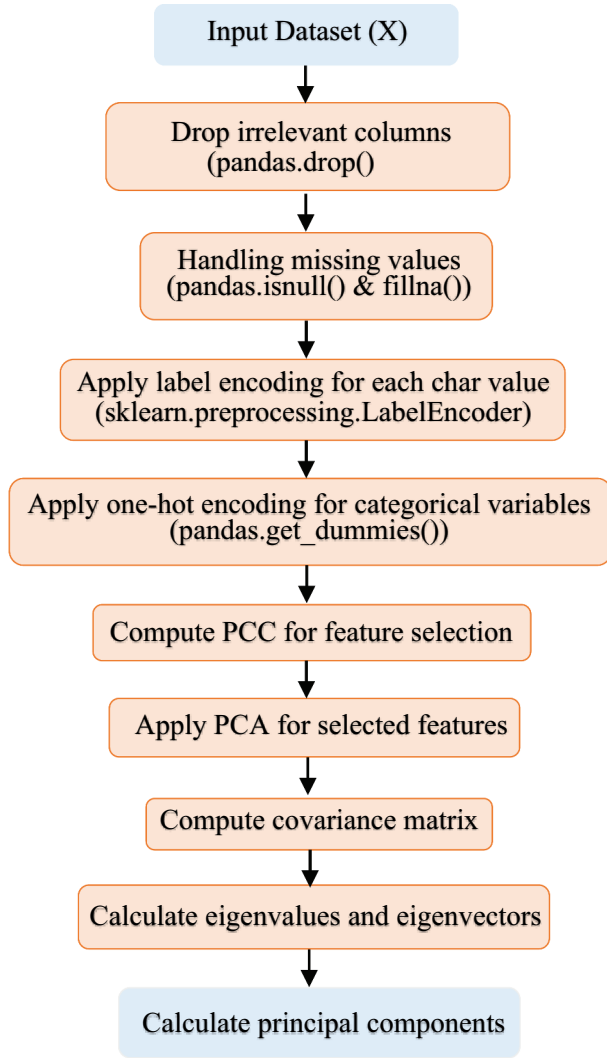


Figure 2. Flow chart of data preprocessing.

if the first component captures the largest variation compared to another component in the resulting data collection.

(Jolliffe, 2005)

Suppose $X(\alpha) = \{r_i, s_i\}_{i=1}^m$ $X(\alpha) = \{r_i, s_i\}$ for

$\alpha = 1, 2, 3, \dots, k$ is the data with zero mean and attributes values, r_i, s_i are data sample and labels of target class. So, the $X(\alpha)$ co-variance matrix is calculated as from equation 2:

$$C = \frac{1}{k-1} \sum_{\alpha=1}^k \{X(\alpha)X(\alpha)^T\} \quad (2)$$

The linear transformation $Y(\alpha)$ is calculated through $X(\alpha)$ by using equation 3.

$$Y(\alpha) = S^T X(\alpha) \quad (3)$$

By utilizing Equation (4), we can address the eigenvalue problem, where S represents an orthogonal matrix with the i^{th} column T equal to the i^{th} covariance matrix.

$$\lambda_i s_i = T s_i \quad (4)$$

λ_i is the eigenvalue of T and eigenvector for this is s_i . For i^{th} principal component, $Y_i(\alpha)$ is calculated from equation (5),

$$Y_i(\alpha) = s_i^T X(\alpha), i = 1, 2, \dots, k \quad (5)$$

To perform feature transformation, the k eigenvectors are arranged in descending order based on the corresponding λ_i eigenvalues. The projection of the new sample $Y(\alpha)$ into the principal space is determined using equation (6).

$$Y(\alpha) = \sum_{i=1}^n c_i^T Y(\alpha) c_i \quad (6)$$

The projection error err of $Y(\alpha)$ is calculated by distance between $Y(\alpha)$ and $Y^{(\alpha)}$.

$$err = \text{dis} \left(Y(\alpha), Y^{(\alpha)} \right) \quad (7)$$

The abovementioned equations are used as a function in our proposed framework to change the optimized dataset into a new encoded state. This transformation is designed to mitigate the risk of inference attacks within the proposed model.

For an example of PCA integration with PCC, consider a dataset with three features: X_1 , X_2 , and X_3 , and a target variable Y . We aim to choose the features high correlated with the target variable using PCC, and then apply PCA to transform these selected features.

- **Compute PCC:** Calculate PCC between each feature and the target variable. Suppose, $PCC(X_1, Y) = 0.85$, $PCC(X_2, Y) = 0.60$, $PCC(X_3, Y) = .90$. Then we select the features with the highest PCC values. In this case, X_1 and X_3 .
- **Apply PCA:** Transform the selected features (X_1 and X_3) into principal components. The resulting principal components PC_1 and PC_2 capture the maximum variance in the data while maintaining the data's privacy.

The combination of PCC and PCA in our approach not only reduced dimensionality and computational complexity but also maintained and highlighted the importance of key features in the intrusion detection task. By examining the loadings of the principal components, we ensured that the model's decisions could be interpreted in terms of the original features, enhancing the overall interpretability of our intrusion detection system. This approach offers valuable insights into the data and helps in understanding the underlying factors driving the model's predictions, thereby making the IoT network more secure and connected.

Algorithm 1: Data Preprocessing and Preparation

Input: Dataset (X)

$X = (X_1, X_2, X_3, X_4, \dots, X_k)$

1. Drop irrelevant columns, Remove NaN
2. Apply label encoding for each char value
3. Apply one-hot encoding for categorical variables
4. Compute PCC from equation 1.

$$PCC(f_1, f_2) = \frac{\sum_{i=1}^n (x_i - \bar{f}_1)(y_i - \bar{f}_2)}{\sqrt{\sum_{i=1}^n (x_i - \bar{f}_1)^2} \sqrt{\sum_{i=1}^n (y_i - \bar{f}_2)^2}}$$

5. Apply PCA on selected k features after PCC
 - i. Compute co-variance matrix by equation (2)

$$C = \frac{1}{k-1} \sum_{a=1}^k \{X(a)X(a)^T\}$$

- ii. Calculate eigen value λ_i and eigen vector s_i from eq. 3 and 4.

$$\lambda_i s_i = T s_i$$

- iii. Calculate i^{th} principal component by equation (5)

$$Y_i(a) = s_i^T X(a), i = 1, 2, \dots, k$$

6. Return Preprocessed and Transformed data (Y)
-

Algorithm 2: Reshaped Data for CNN models

Input: Encoded data (Y)

1. Apply numpy reshape function
 //reshaped training and validation data for 1 D model
 $Y_1 = Y.\text{reshape}(\text{n_sample}, \text{n_component}, 1)$
 //reshaped training and validation data for 2 D model
 $Y_2 = Y.\text{reshape}(\text{n_sample}, \text{height}, \text{width}, 1)$
 //reshaped training and validation data for 3 D model
 $Y_3 = Y.\text{reshape}(\text{n_sample}, \text{depth}, \text{height}, \text{width}, 1)$
 2. Apply train test split function to the reshaped data
 $Y_i = \{(X_{tr}, X_{te}), (Y_{tr}, Y_{te})\}$
 3. Return Reshaped Training and Validation Data (Y')
-

3.2. Deep learning models

DL, a subfield of ML, draws inspiration from the neurobiology of the human brain and its constituent units known as neurons. The three subtypes of DL are supervised, unsupervised, and hybrid. The computational architecture offered by DL techniques combines several processing levels to enable the learning of data representations with multiple layers of abstraction. As a result, DL approaches have substantially enhanced state-of-the-art applications compared to traditional ML approaches.

Algorithm 3: Pseudo code of proposed CNN models

Input: X_{tr} , Y_{tr} = Training data Features and labels

X_{te} , Y_{te} = Training data Features and labels

1. Initialize the model's architecture (number of layers, filters, kernel size, etc.)
 2. Initialize the model's weights and biases.
 3. Define the loss function and optimizer.
 4. For each epoch:
 - i. Iterate through each training example in the X_{tr} , Y_{tr} :
 - a. Pass the example through CNN model.
 - b. Calculate the error between model's prediction and the true label (Y_{tr}).
 - c. Backpropagate the error and update model's weights and biases.
 5. Test the model's accuracy on the validation dataset (X_{te} , Y_{te}).
 6. If the accuracy is satisfactory, return the trained model.
 7. If accuracy is not satisfactory, repeat steps 4–6 with different architecture or hyperparameters.
 8. Return Trained CNN Model
-

Convolutional neural networks (CNNs): A CNN encompasses three layers: convolutional, pooling, and fully connected. Convolutional layers convolve data parameters using different equal-sized filters (kernels). The convolutional layer's primary responsibilities are feature extraction and feature mapping. Additionally, it minimizes the input image size, lowering the procedure's computing expense. The pooling layers use average or maximum pooling to perform downsampling to mitigate the next levels' size. Average pooling combines the values within each cluster from the preceding layer by computing their average. In contrast, max pooling separates the input into several non-overlapping groups and chooses the highest value within each group from the prior layer (Pang et al., 2018). The first layer of a CNN can be a convolution layer, then an activation function, a pooling layer, and finally, a fully connected layer that looks like a feed-forward neural network layer.

We apply convolutional products at the convolutional layer, applying various filters to the input and then an activation function conv . The output $x^{[l]}$ with size $n_H^{[l]}, n_W^{[l]}, n_C^{[l]}$ is shown in equation (8), where $x^{[l-1]}$ is the input with size $(n_H^{[l-1]}, n_W^{[l-1]}, n_C^{[l-1]})$. $n_c^{[l]}$ is the amount of filters, $b_n^{[l]}$ is the bias of the n^{th} convolution, $\psi^{[l]}$ is the activation function, $f^{[l]}$ denotes the size of pooling filter and $D^{(n)}$ has the dimension $(f^{[l]}, f^{[l]}, n_C^{[l-1]})$.

$$x^{[l]} = [\psi^{[l]}(\text{conv}(x^{[l-1]}, D^{(1)}))],$$

$$\psi^{[l]}(\text{conv}(x^{[l-1]}, D^{(2)})) \psi^{[l]}(\text{conv}(x^{[l-1]}, D^{n_C^{[l]}})) \quad (8)$$

$$\text{conv}(x^{[l-1]}, D^{(n)})_{x,y} = \psi^{[l]} \left(\sum_{\alpha=1}^{n_H^{[l-1]}} \sum_{\beta=1}^{n_W^{[l-1]}} \sum_{\gamma=1}^{n_C^{[l-1]}} D_{\alpha,\beta,\gamma}^{(n)} x_{x+\alpha-1,y+\beta-1,\gamma}^{[l-1]} + b_n^{[l]} \right) \quad (9)$$

where $n \in [1, 2, 3, \dots, n_C^{[l]}]$

The pooling layer can be asserted in equation (10),

$$\text{pool}(x^{[l-1]})_{x,y,z} = \phi^{[l]} \left(\left(a_{x+\alpha-1,y+\beta-1,z}^{[l-1]} \right)_{(\alpha,\beta \in 1,2,3,\dots,f^{[l]})^2} \right) \quad (10)$$

The fully connected layer which take in input a vector $x^{[i-1]}$ and return a vector $x^{[i]}$. $w_{\beta,l}$ is the weight. Considering the j^{th} node of the i^{th} layer we have the following equation;

$$z_{\beta}^{[\alpha]} = \sum_{l=1}^{n_{\alpha-1}} w_{\beta,l}^{[\alpha]} x_l^{[\alpha-1]} + b_{\beta}^{[\alpha]} \rightarrow x_{\beta}^{[\alpha]} = \psi^{[\alpha]} \left(z_{\beta}^{[\alpha]} \right) \quad (11)$$

In this approach, we have proposed 1D, 2D, and 3D CNN architectures specifically personalized for intrusion detection in IoT networks. The suggested architectures encompass an initial layer for input, four groups of convolutional layers, a flatten layer, a fully connected layer, and an output layer. Each group is composed of convolutional layers, normalization layers, pooling layers, and dropout layers. Furthermore, we have implemented a reshaping system to preprocess and convert network data into a suitable format for our proposed CNN models (CNN-1D, CNN-2D, and CNN-3D).

3.2.1. CNN-1D

CNN1D model is appropriate for processing sequential data, such as time-series or one-dimensional sensor data commonly found in IoT networks. It accepts one-dimensional input data representing network traffic. In a 1D convolution, calculations are performed along a single dimension, with the kernel moving in one direction. For CNN1D, both the input and output data are 2D, typically used for time-series data. An input vector of 16×1 is created to represent the 16 top features selected by a feature selection algorithm. The model includes four convolutional blocks for feature learning. These layers extract and identify features within the input data, preserving vector relationships. The first convolution layer uses ReLU activation, 8 filters, 3 kernel size, and same padding, resulting in an output of $(16, 8)$. Layer normalization adjusts the activations for each sample in a batch. After this, average pooling reduces the feature maps, outputting $(8, 8)$. A spatial dropout layer with a drop rate of 0.05 is used to prevent overfitting. Each subsequent convolution layer doubles the number of filters. The classification section includes flatten and dense layers, with the flatten layer reshaping the tensor and the dense layer having 64 neurons. The output layer's neuron count corresponds to the number of classes.

3.2.2. CNN-2D

The CNN2D model processes image data, with an input of $4 \times 4 \times 1$ representing a 4×4 grayscale image. The architecture contains of an input layer, four convolutional blocks, fully-connected layers, and an output layer. Each convolutional block includes a convolutional layer with a kernel size of 3×3 , using same padding and ReLU activation. The first block has 8 filters, producing an output of $4 \times 4 \times 8$, followed by normalization, average pooling (pool size 2×2 , stride 2) reducing the output to $2 \times 2 \times 8$, and spatial dropout with a rate of 0.05. The second convolutional block doubles the filters to 16, resulting in $2 \times 2 \times 16$, followed by the same normalization and pooling, reducing the output to $1 \times 1 \times 16$. The third block further doubles the filters to 32, with the output $1 \times 1 \times 32$, applying normalization and dropout but

no pooling. The fourth block doubles the filters again to 64, resulting in $1 \times 1 \times 64$ with normalization. The convolutional blocks are followed by a flatten layer transforming the output into a 64-element vector. This vector is passed to a dense layer with 64 neurons, typically using ReLU activation, and finally to an output layer with a neuron count equal to the number of classes in the dataset, typically using softmax activation for classification.

3.2.3. CNN-3D

The CNN3D model processes 3D image data with an architecture comprising an input layer, four convolutional blocks, a flatten layer, a dense layer, and an output layer. The input is a 3D image with dimensions $4 \times 2 \times 2 \times 1$. The first convolutional layer uses a $3 \times 3 \times 3$ kernel, 8 filters, ReLU activation, and same padding, resulting in an output of $4 \times 2 \times 2 \times 8$, followed by normalization and average pooling with a $2 \times 2 \times 2$ pool size, reducing it to $2 \times 1 \times 1 \times 8$. A spatial dropout layer with a 0.05 dropout rate is added to prevent overfitting. The second block uses a similar structure with a $3 \times 3 \times 3$ kernel, 16 filters, normalization, average pooling, and spatial dropout, reducing the dimensions to $1 \times 1 \times 1 \times 16$. The third block uses a $3 \times 3 \times 3$ kernel, 32 filters, and includes normalization and spatial dropout, keeping dimensions at $1 \times 1 \times 1 \times 32$. The fourth block, with a $3 \times 3 \times 3$ kernel and 64 filters, also includes normalization and spatial dropout, maintaining dimensions at $1 \times 1 \times 1 \times 64$. After the convolutional blocks, a flatten layer reshapes the tensor to a vector of size 64. This vector goes to a dense layer with 64 neurons and then to an output layer, with neurons matching the number of classes, using softmax or sigmoid activation for classification. This architecture captures and processes 3D features effectively, using convolutional layers for feature extraction, pooling for dimensionality reduction, and dense layers for classification.

Each CNN architecture (1D, 2D, and 3D) was chosen based on its suitability for handling the specific characteristics of the input data in IoT networks, such as sequential, spatial, and spatio-temporal features. These architectural choices aim to improve the detection capabilities of the intrusion detection framework by effectively capturing

relevant patterns and reliance in the network traffic data. The pseudocode of our proposed CNN model is presented in Algorithm 3. The convolutional layer architecture of the proposed CNN model is depicted in Figure 3, and the input parameters of the proposed CNN models are depicted in Table 2. The architecture of these models comprises four blocks of convolutional layers: a flatten layer, a dense layer, and an output layer. The four convolutional layer blocks are used to learn features, and they retain the vector relationship while extracting features from the input image. The convolutional layers in this model can extract features starting small data samples inside the input. All the four convolutional layers share the same parameters, excluding the filters, which have replicated in every successive layer. The flatten and dense layers are fully connected and responsible for the classification aspect of the model. The model is flattened by applying the flatten layer, which converts the tensor into a shape that can be compared to the tensor elements. The flatten layer is associated to a dense layer with full connectivity, which is then connected to the output layer. We used a pooling layer in all blocks of the CNN1D model, but it was not deployed in the fourth block of the CNN2D model and subsequently not used in the third and fourth blocks of the CNN3D model. The two-dimensional input and output data were used in the CNN1D model, three-dimensional data in the CNN2D model, and four-dimensional data in the CNN3D model. In CNN1D, the kernel moves in a single direction, whereas in CNN2D, it moves in two directions, and in CNN3D, the kernel navigates in three different orientations. A spatial dropout layer is added after the initial convolution layer to avert overfitting and to regularize the output of the training data.

3.3. Model tuning

DL networks typically require numerous training iterations to converge. However, precise parameter configurations can reduce these iterations by enhancing convolution during training and conducting the network structure. Regularization helps prevent overfitting, and we employed three techniques along with different hyperparameters

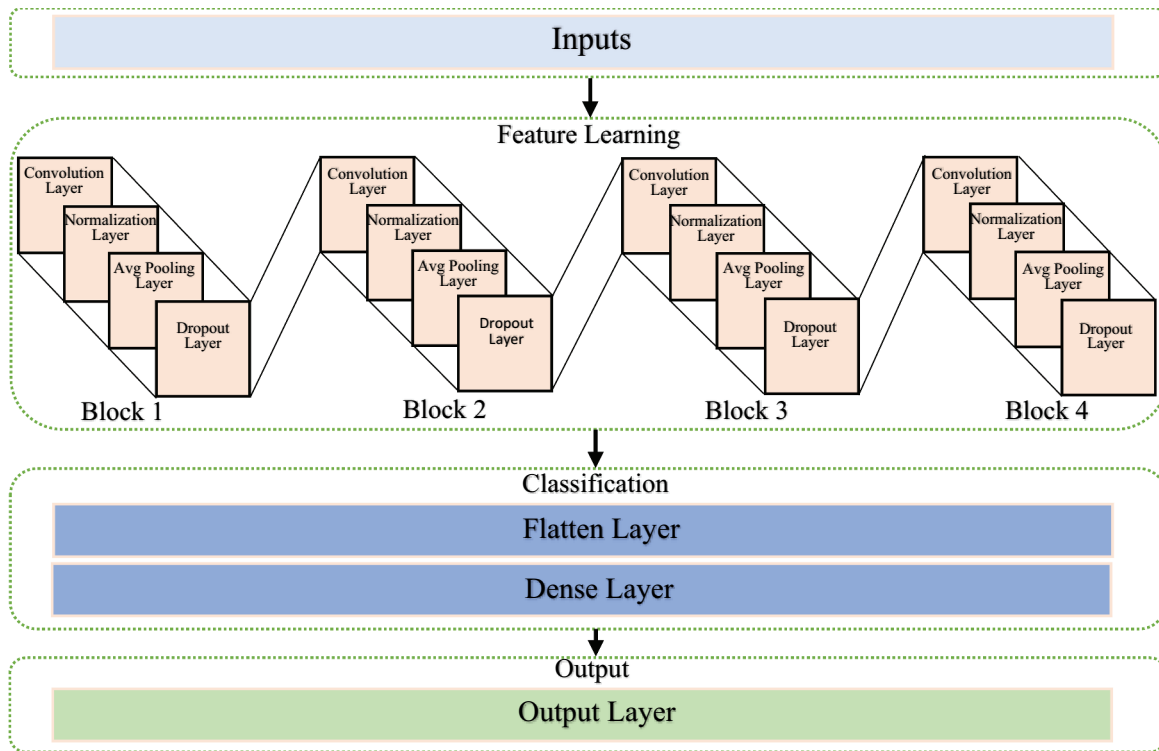


Figure 3. Convolution layer's architecture.

Table 2. DL Model related parameters and architecture hyper-parameters.

		CNN1D	CNN2D	CNN3D
Input Layer	Inputs	16,1	4,4,1	4,2,2,1
Block 1	Convolution Layer	16,8	4,4,8	4,2,2,8
	Normalization Layer	16,8	4,4,8	4,2,2,8
	Ave Pooling Layer	8,8	2,2,8	2,1,1,8
	Dropout Layer	8,8	2,2,8	2,1,1,8
Block 2	Convolution Layer	8,16	2,2,16	2,1,1,16
	Normalization Layer	8,16	2,2,16	2,1,1,16
	Ave Pooling Layer	4,16	1,1,16	1,1,1,16
	Dropout Layer	4,16	1,1,16	1,1,1,16
Block 3	Convolution Layer	4,32	1,1,32	1,1,1,32
	Normalization Layer	4,32	1,1,32	1,1,1,32
	Ave Pooling Layer	2,32	—	—
	Dropout Layer	2,32	1,1,32	1,1,1,32
Block 4	Convolution Layer	2,32	1,1,64	1,1,1,64
	Normalization Layer	2,32	1,1,64	1,1,1,64
	Ave Pooling Layer	1,32	—	—
	Dropout Layer	1,32	1,1,64	1,1,1,64
Kernel Size		3	3,3	3,3,3
Dropout		0.05	0.05	0.05
Fully Connected Layer		64	64	64
Output Layer		5	5	5

for tuning multiclass and binary models. Both models shared hyperparameters and optimization monitoring for generalization across classification problems. Our baseline model was a multiclass CNN with four convolutional blocks, a flatten layer, a dense layer with 64 neurons, and an output layer with neurons representing dataset classes. Layers initiated randomly to learn features over

time. We applied three regularization methods: L1, L2, and dropout. L1 regularization was randomized, and L2 was combined with L1. These regularizations were used on kernel, bias, and activity levels, producing a more generalized model.

We experimented with three CNN architectures, varying convolutional blocks, filters, kernel sizes, and dropout rates. The reference model

performed best. We employed Adam optimizer and sparse categorical cross-entropy loss. The learning rate is crucial for model iteration steps, and after testing various rates, we found 0.003 to be optimal, yielding the highest detection rate. As network learns, loss inversely correlates with output, reducing error and increasing accuracy. To avoid overfitting, we applied early stopping, ceasing training once the validation loss ceased to decrease. We determined 100 epochs as optimal since all models converged within this period, ensuring maximum accuracy.

The activation function is crucial in DL algorithms. ReLU is used in convolution and dense layers, while Softmax is applied in the output layer. Batch size, a pivotal hyperparameter, influences the degree of parallel computation and distribution across worker nodes, potentially speeding up model training. However, larger batch sizes can lead to poor generalization despite similar training losses. The generalization gap represents the difference among training and testing errors. Experiments on batch sizes of 16, 32, 64, and 128 indicated that the optimal range for training and testing the CNN model lies between 64 and 128.

3.3. Model tuning

DL networks typically require numerous training iterations to converge. However, precise parameter configurations can reduce these iterations by enhancing convolution during training and conducting the network structure. Regularization helps prevent overfitting, and we employed three techniques along with different hyperparameters for tuning multiclass and binary models. Both models shared hyperparameters and optimization monitoring for generalization across classification problems. Our baseline model was a multiclass CNN with four convolutional blocks, a flatten layer, a dense layer with 64 neurons, and an output layer with neurons representing dataset classes. Layers initiated randomly to learn features over time. We applied three regularization methods: L1, L2, and dropout. L1 regularization was randomized, and L2 was combined with L1. These regularizations were used on kernel, bias, and activity levels, producing a more generalized model.

We experimented with three CNN architectures, varying convolutional blocks, filters, kernel sizes, and dropout rates. The reference model performed best. We employed Adam optimizer

and sparse categorical cross-entropy loss. The learning rate is crucial for model iteration steps, and after testing various rates, we found 0.003 to be optimal, yielding the highest detection rate. As network learns, loss inversely correlates with output, reducing error and increasing accuracy. To avoid overfitting, we applied early stopping, ceasing training once the validation loss ceased to decrease. We determined 100 epochs as optimal since all models converged within this period, ensuring maximum accuracy.

The activation function is crucial in DL algorithms. ReLU is used in convolution and dense layers, while Softmax is applied in the output layer. Batch size, a pivotal hyperparameter, influences the degree of parallel computation and distribution across worker nodes, potentially speeding up model training. However, larger batch sizes can lead to poor generalization despite similar training losses. The generalization gap represents the difference among training and testing errors. Experiments on batch sizes of 16, 32, 64, and 128 indicated that the optimal range for training and testing the CNN model lies between 64 and 128.

4. Experiment & result

4.1. Dataset

The Edge-IIoTset and NSL-KDD datasets were employed to assess the efficacy of the proposed approach. The Edge-IIoTset is a large dataset that depicts real cyber security scenarios in IoT and IIoT applications (Ferrag et al., 2022). It consists of samples obtained through a seven-layer IoT and IIoT testbed, with every sample containing 61 features collected across multiple connectivity protocols. The dataset comprises two categories of traffic: benign and attack class. The attack traffic consists of 14 distinct attack types that target IoT/IIoT communication protocols. These attacks result in five threat classes: malware, DDoS, injection, scanning, and MITM. The dataset contained 1,09,304 samples of benign and attack traffic.

The NSL-KDD dataset contains 41 attributes, including protocol type, service, duration, source bytes, flag, etc (Tavallae et al., 2009). It is divided into five distinct attack classes: Normal class, Denial of Service (DoS), Probe, User to Root

Table 3. Binary classification results.

Model	Edge-IIoTset				NSL-KDD			
	Accuracy	Precision	Recall	F1 Score	Accuracy	Precision	Recall	F1 Score
CNN-1D	99.75	99.80	99.79	99.84	99.11	97.26	97.52	97.64
CNN-2D	99.89	99.88	99.92	99.98	99.33	98.44	98.68	98.80
CNN-3D	99.66	99.70	98.96	99.73	99.17	98.52	97.65	96.69

(U2R), and Remote to Local (R2L). It consists of 125,973 training and 22,544 testing records. The training dataset comprises 67,343 normal class samples 45,927 DoS class samples 11,656 Probe class samples, 995 R2L class samples, and 52 U2R class samples. The testing dataset includes 9711 samples of the normal class, 7458 samples of DoS, 2754 Probe samples, 2421 R2L samples, and 200 U2R samples (Hong et al., 2021).

4.2. Experimental setup

The proposed system was implemented using Python 3.10. On the backend, the Keras API (version 2.9.0) operated atop the TensorFlow library (version 2.9.1). The experiment conducted on a system with a Windows-10 Pro 64-bit operating system, featuring an Intel Xeon E5-2670 processor operating at 2.60 GHz, 16 GB of RAM, and an NVIDIA-QUADRO-P4000 graphics card with 12 GB of storage. The model commenced with uniform distribution initialization, while optimization utilized the Adam optimizer with a learning rate of 0.003. Assessment utilized a batch size of 128, with dropout rates set to 0.05. To ensure reproducibility, we set the random seed values to 42, using `numpy.random.seed(42)`, `tensorflow.random.set_seed(42)`, and `random.seed(42)`. Additionally, we used specific software versions for our implementation, including Numpy 1.21.6, Pandas 1.4.2, Scikit-Learn 1.1.1, and Matplotlib 3.5.2 for visualization.

4.3. Evaluation metrics

The effectiveness of our model is assessed using the following assessment metrics in this study.

- Accuracy: The correct prediction is known as Accuracy.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision: Precision is defined as the proportion of correctly predicted positive outcomes to total positive predictions.

$$Precision = \frac{TP}{TP + FP}$$

- Recall: The ratio of correctly predicted positive outcomes to total outcomes is referred to as recall.

$$Recall = \frac{TP}{TP + FN}$$

- F1-Score: The mean of precision and recall is referred as F1 Score. F1 Score gives output between a range from 0 to 1.

$$F1\ Score = \frac{2 * (Recall * Precision)}{Recall + Precision}$$

5. Result

After applying PCC, we select only 19 features from the given dataset. We further reduce the dimensionality using PCA and then reshape the dataset. These reshaped data were used as input to train the 1D, 2D, and 3D-CNN models. The proposed CNN models were trained and evaluated for binary and multi-class classifications. The dataset consists of attacks with distinct patterns and was used to assess the CNN model's ability to detect these patterns from the raw data. The highest-performing CNN model and the results of binary and multi-class classifications of the proposed model are discussed below.

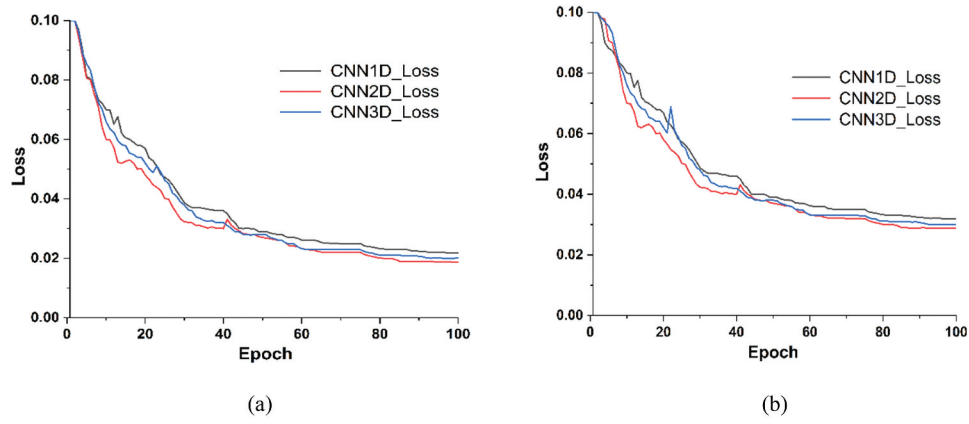


Figure 4. Loss trends in binary class classification on (a) edge-IIoTset dataset and (b) NSL-KDD dataset.

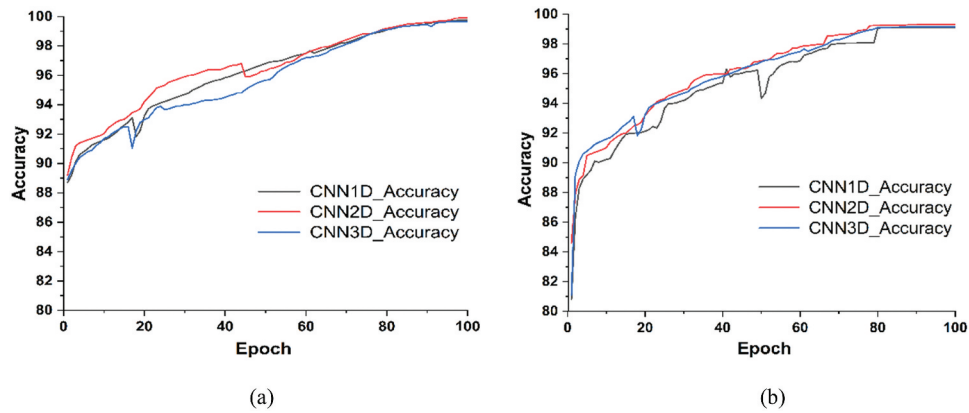


Figure 5. Accuracy trends in binary class classification on (a) edge-IIoTset dataset and (b) NSL-KDD dataset.

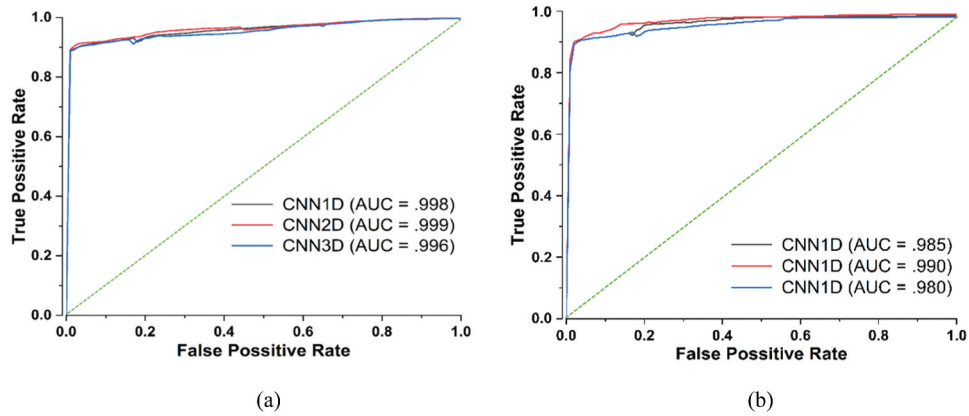


Figure 6. ROC curve of proposed models on (a) edge-IIoTset dataset and (b) NSL-KDD dataset.

5.1. Binary class classification

We assessed the efficacy of the proposed model using both datasets and measured their performance with metrics including accuracy, recall, precision, and the F1 score. Our binary classification models utilized a common set of features and

attained an average accuracy of 99.76% on the Edge-IIoTset dataset and 99.20% on the NSL-KDD dataset. Binary classification categorizes each dataset instance as either a benign or an intrusion network packet. By employing the PCA-PCA technique, we effectively addressed the issue of

Table 4. Multiclass classification results for edge-IloTset dataset.

Model	CNN-1D				CNN-2D				CNN-3D			
Class	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
Benign	99.52	99.70	99.10	99.34	99.65	99.60	99.70	99.59	99.56	99.68	99.78	99.65
DDoS	99.24	99.70	99.60	99.34	99.30	99.26	99.60	99.92	99.10	99.64	98.88	99.90
Injection	99.45	98.93	99.90	99.01	99.54	99.09	99.66	98.91	99.09	99.62	98.90	98.92
Scanning	99.32	99.10	98.60	99.09	99.62	98.66	99.33	99.40	99.40	99.51	98.08	99.10
Malware	99.34	98.33	96.62	99.15	99.60	94.28	98.44	96.22	99.52	99.86	98.90	98.52
MITM	99.25	96.05	93.80	94.20	99.54	97.44	94.08	95.20	99.40	99.42	97.80	98.50

Table 5. Multiclass classification results for NSL-KDD dataset.

Model	CNN-1D				CNN-2D				CNN-3D			
Class	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
Normal	99.02	98.50	96.60	98.84	99.11	98.20	97.02	98.69	99.04	97.90	98.20	99.10
DoS	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
Probe	89.65	95.23	90.90	93.01	90.34	94.39	92.66	93.51	81.48	87.62	78.70	82.92
R2L	98.02	99.40	98.60	99.49	99.25	98.66	99.33	99.48	98.50	98.31	100.00	99.05
U2R	74.94	68.33	78.62	73.05	80.32	85.00	88.97	86.91	77.83	76.66	86.92	81.42

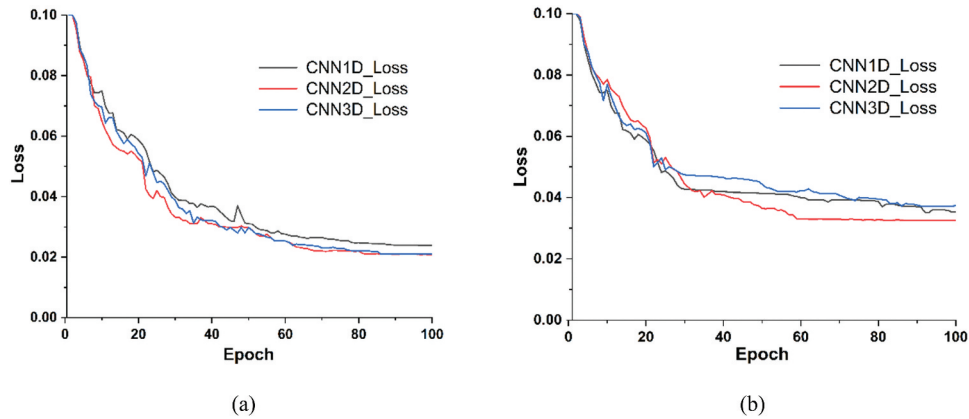
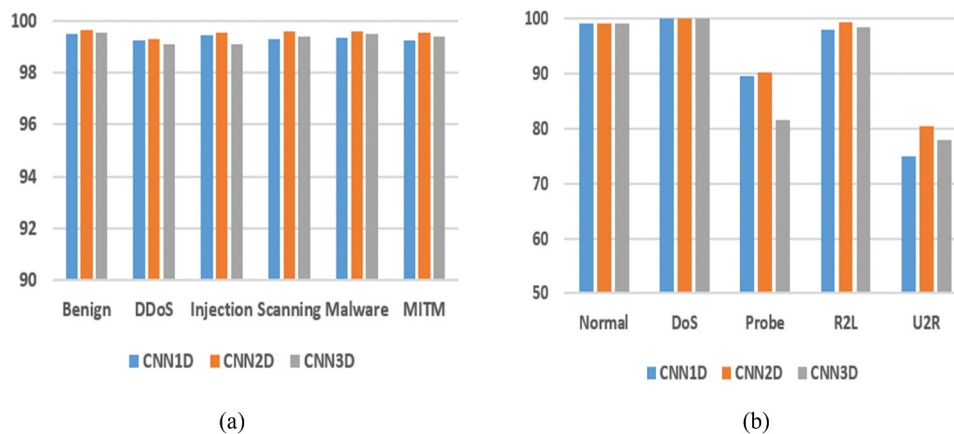
**Figure 7.** Loss trends in multi-class classification on (a) edge-IloTset dataset and (b) NSL-KDD dataset.**Figure 8.** CNN Model's accuracy in multi-class classification on (a) edge-IloTset dataset and (b) NSL-KDD dataset.

Table 6. Performance comparison with recent works on edge-IIoTset dataset.

Authors	DL/ML Models	Accuracy (%)	Precision (%)	Recall (%)
Javeed et al. (2023)	BiGRU-LSTM	98.32	98.78	97.22
Bhandari et al. (2023)	DNN	94.00	98.00	89.00
Proposed Model	PCA-CNN	99.76	99.79	99.89

Table 7. Performance comparison with recent works on NSL-KDD dataset.

Authors	DL/ML Models	Accuracy (%)	Precision (%)	Recall (%)
Barnard et al. (2022)	XGBoost	93.28	91.05	97.81
Fatani, Dahou, et al. (2021)	AQU	77.38	83.69	77.38
Imrana et al. (2021)	BiLSTM	91.36	92.81	91.36
	SVM	77.12	80.80	77.12
	Naïve Bayes	71.48	76.30	71.50
	Random Forest	77.07	82.20	77.10
Rani et al. (2022)	LDA based ELM	85.56	97.09	76.94
Proposed Model	PCA-CNN	99.20	98.07	97.95

overfitting by reducing dimensionality. We implemented a 20-iteration patience and an early stopping approach during the training phase. The validation was concluded within fewer than 100 epochs, significantly reducing validation loss.

Table 3 presents the binary classification results of the proposed model on both datasets. The findings show that the CNN models achieve superior accuracy, precision, recall, and F1 score. Figure 4 illustrates the performance of our proposed binary CNN models in terms of loss versus epochs, while Figure 5 illustrates the relationship between the number of epochs and the accuracy trend of our CNN models. The curves depict inverse functions for the accuracy and loss values. Figure 6 illustrates the ROC curve of our proposed model, which confirms its strong performance by achieving a high AUC (Area Under the ROC Curve) score, representing a very high true positive rate and a low false positive rate.

5.2. Multiclass classification

Tables 4 and 5 demonstrate the performance of the proposed models in multiclass classification, where the dataset was categorized as benign network traffic or any attack class. The average accuracy of the proposed model in multiclass classification is 99.41% on the Edge-IIoTset dataset and 92.43% on the NSL-KDD dataset. Our multiclass models achieved high accuracy across all threat classes and one benign class. This research demonstrates that

these models can effectively classify different cyber-attacks observed in datasets or real IoT networks.

Figure 7 depicts the performance of the CNN models in multiclass classification concerning loss versus accuracy, while Figure 8 presents a bar chart of the models' accuracy. The above results indicate that the benign or normal class achieved the highest performance in all models. It can be observed that the loss is minimized when the number of epochs reaches 100.

6. Discussion and comparison

Tables 6 and 7 present the performance comparison of the proposed PCA-CNN approach with other state-of-the-art techniques, evaluated on the Edge-IIoTset and NSL-KDD datasets. The results show that the proposed CNN models consistently outperform existing ML/DL models in terms of accuracy, precision, and recall. The proposed models demonstrate superior performance in both binary and multiclass classification tasks, achieving remarkable average accuracy rates on both datasets. The CNN-2D model shows the highest accuracy among the three CNN models. However, all three models outperform traditional ML-based methods, highlighting the effectiveness of our approach.

The integration of PCA and PCC in our framework addresses feature selection and dimensionality reduction effectively. PCA preserves essential information in a lower-dimensional space, while PCC selects features based on their correlation,

retaining those most pertinent to the detection task. This combination enhances data representation and the model's discriminative power. PCA also mitigates overfitting, while PCC boosts generalization, resulting in improved model performance. The reduced feature space speeds up the training of CNN models, improving computational efficiency without compromising detection accuracy. The proficient feature extraction by CNN models, combined with the dimensionality reduction achieved by PCA, contributes significantly to computational efficiency. The reduced feature space leads to faster training and testing times, as observed on both datasets. This approach balances model complexity and performance, ensuring only the most relevant features are fed into the CNN, reducing overfitting and unnecessary complexity. The framework effectively handles larger datasets and more complex IoT environments, indicating good scalability.

Although hyperparameter selection might still impact system performance, our results suggest that the proposed approach achieves high accuracy without overfitting. Compared to previous models, our framework demonstrates considerable gains in accuracy, precision, recall, and F1-score by integrating PCC-PCA for feature selection and dimensionality reduction with CNN for classification. Encoding data into various dimensions improves the model's ability to detect intrusions accurately. The evaluation on both datasets confirms the framework's superiority, exhibiting faster testing times and higher detection rates.

In summary, the proposed PCA-CNN framework provides a novel and effective approach for securing IoT networks. It combines enhanced performance metrics with computational efficiency, model complexity, and scalability, contributing to a safer interconnected world. Future work will include explicit measurements of computational efficiency and a deeper exploration of model complexity and scalability.

7. Conclusion & future work

In conclusion, our research findings demonstrate a new approach to detecting intrusions in IoT networks by combining PCA and CNN. We evaluated

the effectiveness of our model on the two datasets. To address the issue of the dataset's class imbalance, we utilized PCC-PCA for feature selection by reducing its dimensionality. Our approach employs the power of 1D-CNN, 2D-CNN, and 3D-CNN models to identify and categorize binary and multi-class intrusions. The strength of CNN models is their capability to learn and identify patterns in network traffic data, resulting in higher accuracy and adaptability. We demonstrated that our proposed method outperforms existing state-of-the-art intrusion detection methods through extensive examinations, as proven by considerable increases in accuracy, recall, precision, detection rate, and F1 score. Notable improvements in all these metrics evidenced the result.

In the future, we envision applying transfer learning techniques to expand the capabilities of our model and enable intrusion detection across multiple datasets. This extension can further improve the effectiveness and performance of the proposed approach. By continuously advancing the field of cybersecurity, our novel CNN-based intrusion detection method contributes to reinforcing the security of IoT networks and addressing the growing security concerns in contemporary living.

Acknowledgments

The authors acknowledge that they received no funding for this study.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Compliance with ethical standards

The authors of this study have not conducted any research with humans or animals.

Data availability statement

Data will be provided upon request.

References

- Abiodun, I. O., Abiodun, E. O., Alawida, M., Alkhawaldeh, R. S., & Arshad, H. (2021). A review on the

- security of the internet of things: Challenges and solutions. *Wireless Personal Communications*, 119(3), 2603–2637. <https://doi.org/10.1007/s11277-021-08348-9>
- Al-Garadi, M. A., Mohamed, A., Al-Ali, A. K., Du, X., Ali, I., & Guizani, M. (2020). A survey of machine and deep learning methods for internet of things (IoT) security. *IEEE Communications Surveys & Tutorials*, 22(3), 1646–1685. <https://doi.org/10.1109/COMST.2020.2988293>
- Almotairi, A., Atawneh, S., Khashan, O. A., & Khafajah, N. M. (2024). Enhancing intrusion detection in IoT networks using machine learning-based feature selection and ensemble models. *Systems Science and Control Engineering*, 12(1), 2321381. <https://doi.org/10.1080/21642583.2024.2321381>
- Alqahani, A., & Khan, S. B. (2023). An optimal hybrid cascade regional convolutional network for cyberattack detection. *International Journal of Network Management*, 34(5), e2247. <https://doi.org/10.1002/nem.2247>
- Barnard, P., Marchetti, N., & DaSilva, L. A. (2022). Robust network intrusion detection through explainable artificial intelligence (XAI). *IEEE Networking Letters*, 4(3), 167–171. <https://doi.org/10.1109/LNET.2022.3186589>
- Bhandari, G., Lyth, A., Shalaginov, A. G., & M, T. (2023). Distributed deep neural-network-based middleware for cyber-attacks detection in smart IoT ecosystem: A novel framework and performance evaluation approach. *Electronics*, 12(2), 298. <https://doi.org/10.3390/electronics12020298>
- Chettri, L., & Bera, R. (2020). A comprehensive survey on internet of things (IoT) toward 5G wireless systems. *IEEE Internet of Things Journal*, 7(1), 16–32. January. 2020. <https://doi.org/10.1109/JIOT.2019.2948888>
- Dahou, A., Abd-Elaziz, M., Chelloug, S. A., Awadallah, M. A., Al-Betar, M. A., Al-Qaness, M. A., & Forestiero, A. (2022). Intrusion detection system for IoT based on deep learning and modified reptile search algorithm. *Computational Intelligence and Neuroscience*, 2022, 1687–5265. <https://doi.org/10.1155/2022/6473507>
- Ding, W., Abdel-Basset, M., & Mohamed, R. (2023). DeepAK-IoT: An effective deep learning model for cyberattack detection in IoT networks. *Information Sciences*, 634, 157–171. <https://doi.org/10.1016/j.ins.2023.03.052>
- Djaidja, T. E. T., Brik, B., Mohammed, S. S., Boualouache, A., & Ghamri-Doudane, Y. (2024). Early network intrusion detection enabled by attention mechanisms and RNNs. *IEEE Transactions on Information Forensics and Security*, 19, 7783–7793. <https://doi.org/10.1109/TIFS.2024.3441862>
- Fatani, A., Abd-Elaziz, M., Dahou, A., Al-Qaness, M. A. A., & Lu, S. (2021). IoT intrusion detection system using deep learning and enhanced transient search optimization. *Institute of Electrical and Electronics Engineers Access*, 9, 123448–123464. <https://doi.org/10.1109/ACCESS.2021.3109081>
- Fatani, A., Dahou, A., Al-Qaness, M. A. A., Lu, S., & Abd Elaziz, M. A. (2021). Advanced feature extraction and selection approach using deep learning and aquila optimizer for IoT intrusion detection system. *Sensors*, 22(1), 140. <https://doi.org/10.3390/s22010140>
- Ferrag, M. A., Friha, O., Hamouda, D., Maglaras, L., & Janicke, H. (2022). Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning. *Institute of Electrical and Electronics Engineers Access*, 10(40), 281–306. <https://doi.org/10.1109/ACCESS.2022.3165809>
- Firdous, S. N., Baig, Z., Valli, C., & Ibrahim, A. (2017). Modelling and evaluation of malicious attacks against the IoT MQTT protocol. *IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)* (pp. 748–755, Exeter, UK. <https://doi.org/10.1109/iThings-GreenCom-CPSCom-SmartData.2017.115>
- Gómez, A., & Muñoz, A. (2023). Deep learning-based attack detection and classification in android devices. *Electronics*, 12(15), 3253. <https://doi.org/10.3390/electronics12153253>
- Guo, Y., Zhou, Y., & Zhang, Z. (2021). Fault diagnosis of multi-channel data by the CNN with the multilinear principal component analysis. *Measurement*, 171, 108513. <https://doi.org/10.1016/j.measurement.2020.108513>
- Han, M. L., Kwak, B. I., & Kim, H. K. (2023). TOW-IDS: Intrusion detection system based on three overlapped wavelets for automotive ethernet. *IEEE Transactions on Information Forensics and Security*, 18, 411–422. <https://doi.org/10.1109/TIFS.2022.3221893>
- Hong, R. F., Horng, S. C., & Lin, S. S. (2021). Machine learning in cyber security analytics using NSL-KDD dataset. *International Conference on Technologies and Applications of Artificial Intelligence (TAAI)* (pp. 260–265, Taichung, Taiwan. <https://doi.org/10.1109/TAAI54685.2021.00057>
- Hussain, F., Hussain, R., Hassan, S. A., & Hossain, E. (2020). Machine learning in IoT security: Current solutions and future challenges. *IEEE Communications Surveys & Tutorials*, 22(3), 1686–1721. <https://doi.org/10.1109/COMST.2020.2986444>
- Imran, M., Haider, N., Shoaib, M., & Razzak, I. (2022). An intelligent and efficient network intrusion detection system using deep learning. *Computers & Electrical Engineering*, 99, 107764. <https://doi.org/10.1016/j.compeleceng.2022.107764>
- Imrana, Y., Xiang, Y., Ali, I., & Rauf, Z. A. (2021). A bidirectional LSTM deep learning approach for intrusion detection. *Expert Systems with Applications*, 185, 115524. <https://doi.org/10.1016/j.eswa.2021.115524>
- Jamalipour, A., & Murali, S. (2022). A taxonomy of machine-learning-based intrusion detection systems for the internet of things: A survey. *IEEE Internet of Things Journal*, 9(12), 9444–9466. <https://doi.org/10.1109/JIOT.2021.3126811>
- Javeed, D., Gao, T., Saeed, M. S., & Kumar, P. (2023). An intrusion detection system for edge-envisioned smart agriculture in extreme environment. *IEEE internet of things journal*. <https://doi.org/10.1109/JIOT.2023.3288544>

- Jayalaxmi, P. L. S., Saha, R., Kumar, G., Conti, M., & Kim, T. H. (2022). Machine and deep learning solutions for intrusion detection and prevention in IoTs: A survey. *Institute of Electrical and Electronics Engineers Access*, 10, 121173–121192. <https://doi.org/10.1109/ACCESS.2022.3220622>
- Jolliffe, I. T. (2005). Principal component analysis. In *Encyclopedia of Statistics in Behavioral Science* (pp. 1–9). Hoboken, NJ, USA: John Wiley & Sons. <https://doi.org/10.1002/0470013192.bsa501>
- Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of intrusion detection systems: Techniques, datasets and challenges. *Cybersecurity*, 2(12). <https://doi.org/10.1186/s42400-019-0038-7>
- Kirmani, S., Mazid, A., Khan, I. A., & Abid, M. (2022). Survey on IoT-enabled Smart grids: Technologies, architectures, applications, and challenges. *Sustainability*, 15(1), 717. <https://doi.org/10.3390/su15010717>
- Kothai, G., & Poovammal, E. (2024). A hybrid CNN-GRU-based intrusion detection system for secure communication in vehicular adhoc network. *Information Security Journal: A Global Perspective*, 1–11. <https://doi.org/10.1080/19393555.2024.2361244>
- Li, Y., Xu, Y., Liu, Z., Hou, H., Zheng, Y., & Xin, Y. (2020). Robust detection for network intrusion of industrial IoT based on multi-cnn fusion. *Measurement*, 154, 107450. <https://doi.org/10.1016/j.measurement.2019.107450>
- Li, Y., Zuo, Y., Song, H., & Lv, Z. (2022). Deep learning in security of internet of things. *IEEE Internet of Things Journal*, 9(22), 22133–22146. <https://doi.org/10.1109/JIOT.2021.3106898>
- Ling, Z., & Hao, Z. J. (2022a). An intrusion detection system based on normalized mutual information antibodies feature selection and adaptive quantum artificial immune system. *International Journal on Semantic Web and Information Systems (IJSWIS)*, 18(1), 1–25. <https://doi.org/10.4018/IJSWIS.308469>
- Liu, Y., & Durllofsky, L. J. (2021). 3D CNN-PCA: A deep-learning-based parameterization for complex geomodels. *Computers & Geosciences*, 148, 104676. <https://doi.org/10.1016/j.cageo.2020.104676>
- Mazid, A., Abid, M., & Kirmani, S. (2023). A hybrid approach based on principal component analysis and convolution neural network for power theft detection. *International Conference on Recent Advances in Electrical, Electronics & Digital Healthcare Technologies (REEDCON)*, New Delhi, India (pp. 313–317). <https://doi.org/10.1109/REEDCON57544.2023.10150839>
- Meffert, C., Clark, D., Baggili, I., & Breitingner, F. (2017). Forensic state acquisition from internet of things (FSAIoT): A general framework and practical approach for IoT forensics through IoT device state acquisition. *Proc. 12th Int. Conf. Availability, Rel. Secur.* (pp. 1–11). <https://doi.org/10.1145/3098954.3104053>
- Mishra, A., Gupta, B. B., & Joshi, R. C. (2011, September). A comparative study of distributed denial of service attacks, intrusion tolerance and mitigation techniques. 2011 European Intelligence and Security Informatics Conference, Athens, Greece (pp. 286–289).
- Muñoz, A., Fernández-Gago, C., & López-Villa, R. (2022). A test environment for wireless hacking in domestic IoT scenarios. *Mobile Networks & Applications*, 28(4), 1–10. <https://doi.org/10.1007/s11036-022-02046-x>
- Ning, J., Gui, G., Wang, Y., Yang, J., Adebisi, B., Ci, S., Gacanin, H., & Adachi, F. (2022). Malware traffic classification using domain adaptation and ladder network for secure industrial internet of things. *IEEE Internet of Things Journal*, 9(18), 17058–17069. <https://doi.org/10.1109/JIOT.2021.3131981>
- Pang, Y., Sun, M., Jiang, X., & Li, X. (2018). Convolution in convolution for network in network. *IEEE Transactions on Neural Networks and Learning Systems*, 29(5), 1587–1597. <https://doi.org/10.1109/TNNLS.2017.2676130>
- Pawar, V., & Sachdeva, S. (2022). CovidBChain: Framework for access-control, authentication, and integrity of Covid-19 data. *Concurrency & Computation: Practice & Experience*, 34(28), 7397. <https://doi.org/10.1002/cpe.7397>
- Popoola, S. I., Adebisi, B., Hammoudeh, M., Gui, G., & Gacanin, H. (2021). Hybrid deep learning for botnet attack detection in the internet of things networks. *IEEE Internet of Things Journal*, 8(6), 4944–4956. <https://doi.org/10.1109/JIOT.2020.3034156>
- Rani, M., & Gagandeep, G. (2022). Effective network intrusion detection by addressing class imbalance with deep neural networks multimedia tools and applications. *Multimedia Tools Appl*, 81(6), 8499–8518. <https://doi.org/10.1007/s11042-021-11747-6>
- Senthil, S., & Muthukumaran, N. (2023). Joined Bi-model RNN with spatial attention and GAN based IoT botnet attacks detection. *Sādhanā*, 48(3), 141. <https://doi.org/10.1007/s12046-023-02188-y>
- Srilatha, D., & Shyam, G. K. (2021). Cloud-based intrusion detection using kernel fuzzy clustering and optimal type-2 fuzzy neural network. *Cluster Computing*, 24(3), 2657–2672. <https://doi.org/10.1007/s10586-021-03281-9>
- Sufyan, F., & Banerjee, A. (2020). Computation offloading for distributed Mobile edge computing network: A multiobjective approach. *Institute of Electrical and Electronics Engineers Access*, 8, 149915–149930. <https://doi.org/10.1109/ACCESS.2020.3016046>
- Tavallae, M., Bagheri, E., Lu, W., & Ghorbani, A. (2009). A detailed analysis of the KDD CUP 99 data set. IEEE Symposium on Computational Intelligence for Security and Defense Applications (Vol. 2009, pp. 1–6, Ottawa, ON, Canada. <https://doi.org/10.1109/CISDA.2009.5356528>
- Venkatesh, B., & Anuradha, J. (2019). A review of feature selection and its methods. *Cybernetics and Information Technologies*, 19(1), 3–26. <https://doi.org/10.2478/cait-2019-0001>
- Wang, Z., Xie, X., Chen, L., Song, S., & Wang, Z. (2023). Intrusion detection and network information security based on deep learning algorithm in urban rail transit

- management system. *IEEE Transactions on Intelligent Transportation Systems*, 24(2), 2135–2143. <https://doi.org/10.1109/TITS.2021.3127681>
- Wei, J., Long, C., Li, J., & Zhao, J. (2020). An intrusion detection algorithm based on bag representation with ensemble support vector machine in cloud computing. *Concurrency Computation, Practice and Experience*, 32(24), 5922. <https://doi.org/10.1002/cpe.5922>
- Xu, C., Shen, J., & Du, X. (2020). A method of few-shot network intrusion detection based on meta-learning framework. *IEEE Transactions on Information Forensics and Security*, 15, 3540–3552. <https://doi.org/10.1109/TIFS.2020.2991876>
- Xu, C., Shen, J., Du, X., & Zhang, F. (2018). An intrusion detection system using a deep neural network with gated recurrent units. *Institute of Electrical and Electronics Engineers Access*, 6, 48697–48707. <https://doi.org/10.1109/ACCESS.2018.2867564>
- Yan, J., Qi, Y., & Rao, Q. (2018). Detecting malware with an ensemble method based on deep neural network. *Security and Communication Networks*, 2018, 1–16. <https://doi.org/10.1155/2018/7247095>
- Zhang, L., Jiang, S., Shen, X., Gupta, B. B., & Tian, Z. (2021). PWG-IDS: An intrusion detection model for solving class imbalance in IIoT networks using generative adversarial networks. arXiv preprint arXiv:2110.03445. <https://doi.org/10.48550/arXiv.2110.03445>
- Zhao, R., Gui, G., Xue, Z., Yin, J., Ohtsuki, T., Adebisi, B., & Gacanin, H. (2022). A novel intrusion detection method based on lightweight neural network for internet of things. *IEEE Internet of Things Journal*, 9(12), 9960–9972. <https://doi.org/10.1109/JIOT.2021.3119055>

Copyright of Information Security Journal: A Global Perspective is the property of Taylor & Francis Ltd and its content may not be copied or emailed to multiple sites without the copyright holder's express written permission. Additionally, content may not be used with any artificial intelligence tools or machine learning technologies. However, users may print, download, or email articles for individual use.